lamae

BUT2 INFORMATIQUE · RESSOURCE R3.05

Programmation système

CM 04 — Les signaux et les IPC

```
bash — R3.05
$ ./serveur &
[1] 4242
$ kill -TERM 4242
SIGTERM reçu : arrêt propre
[1]+  Fini    ./serveur
```

Christophe Cruz · christophe.cruz@ube.fr · 2026-2027

IUT Dijon-Auxerre · Université Bourgogne Europe

Au programme

01

Les signaux

Ce qu'est un signal, d'où il vient, kill, signal, pause et alarm.

02

Intercepter avec sigaction

La structure sigaction, les options, les règles d'un bon gestionnaire.

03

Bloquer des signaux

Ensembles de signaux, sigprocmask, signaux pendants, sigsuspend.

04

Les IPC

Files de messages, sémaphores et mémoire partagée, en System V et en POSIX.

Prérequis : CM 03a (processus, fork, états, première approche des signaux) et CM 03b (tubes).

01

Les signaux

Prévenir un processus qu'un événement s'est produit

Qu'est-ce qu'un signal ?

- Un moyen de communication entre processus sans transfert de données : juste « il s'est passé quelque chose »
- Il est asynchrone : il peut arriver à n'importe quel moment de l'exécution
- Chaque signal a un numéro et un nom symbolique (SIGINT, SIGTERM…)
- Par défaut, le destinataire ne sait pas qui l'a envoyé (sauf avec SA_SIGINFO, partie 02)

les signaux de Linux

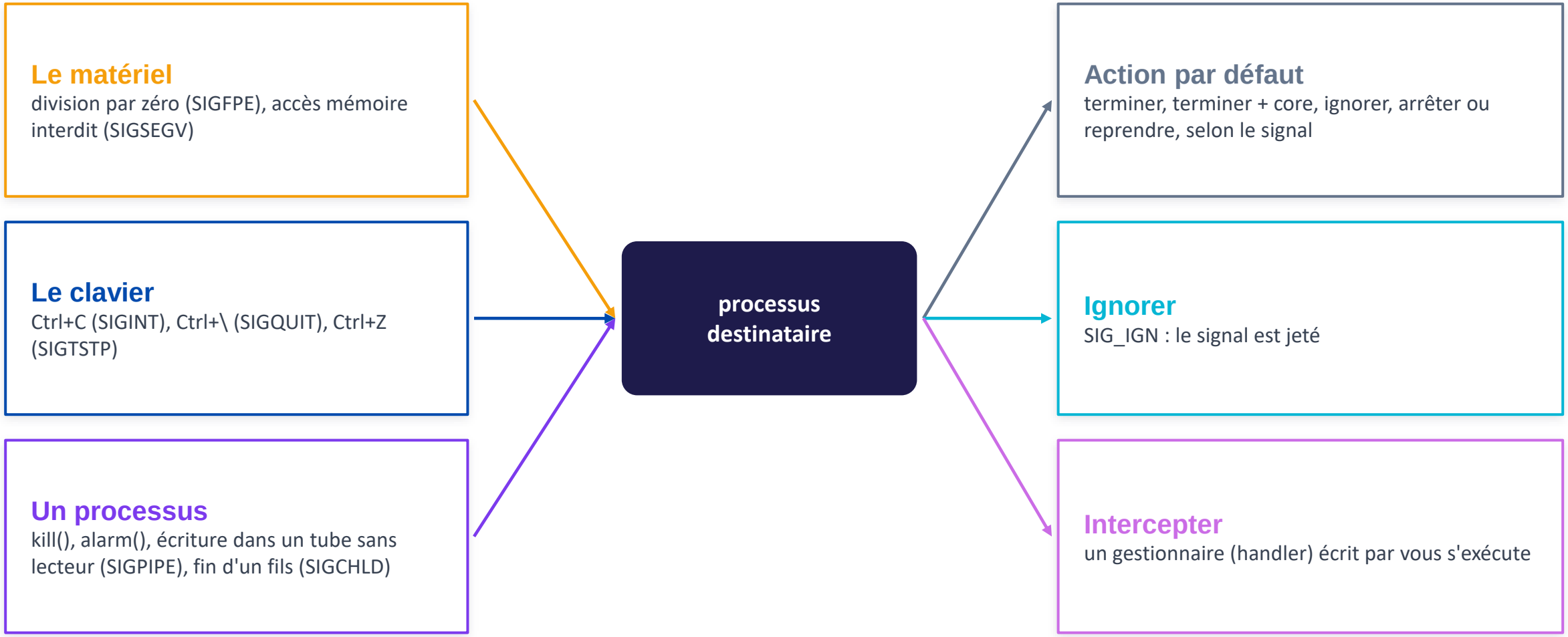
```
$ kill -l
 1) SIGHUP    2) SIGINT    3) SIGQUIT   4) SIGILL
 5) SIGTRAP   6) SIGABRT   7) SIGBUS    8) SIGFPE
 9) SIGKILL  10) SIGUSR1  11) SIGSEGV  12) SIGUSR2
13) SIGPIPE  14) SIGALRM  15) SIGTERM  16) SIGSTKFLT
17) SIGCHLD  18) SIGCONT  19) SIGSTOP  20) SIGTSTP
...
34) SIGRTMIN ... 64) SIGRTMAX
```

Utilisez les noms, pas les numéros : les numéros changent selon l'architecture (SIGSTOP vaut 19 sur x86, 17 sur d'autres). Les noms sont définis dans <signal.h>.

Les signaux à connaître

Signal	Déclencheur	Action par défaut
SIGINT	Ctrl+C	terminer
SIGQUIT	Ctrl+\	terminer + fichier core
SIGTERM	kill (sans option)	terminer
SIGKILL	kill -9	terminer (ni ignorable, ni interceptable)
SIGSTOP / SIGTSTP	kill -STOP / Ctrl+Z	arrêter (état T)
SIGCONT	fg, bg, kill -CONT	reprendre
SIGSEGV	accès mémoire invalide	terminer + core
SIGPIPE	écriture dans un tube sans lecteur	terminer
SIGALRM	fin d'un alarm()	terminer
SIGCHLD	fin ou arrêt d'un fils	ignorer
SIGUSR1 / SIGUSR2	libres pour vos programmes	terminer

D'où viennent les signaux, et que se passe-t-il ?



SIGKILL et SIGSTOP ne peuvent être ni ignorés, ni interceptés.

Envoyer un signal : kill

```c
#include <signal.h>

int kill(pid_t pid, int sig);     // 0 si succès, -1 si erreur
```

pid	Destinataire
`> 0`	le processus de ce PID
`0`	tous les processus de mon groupe
`-1`	tous ceux que j'ai le droit de signaler
`< -1`	tous les processus du groupe |pid|

- On ne peut signaler que ses propres processus (sauf root)
- kill(pid, 0) n'envoie rien : il teste si le processus existe
- La commande kill du shell utilise cet appel système

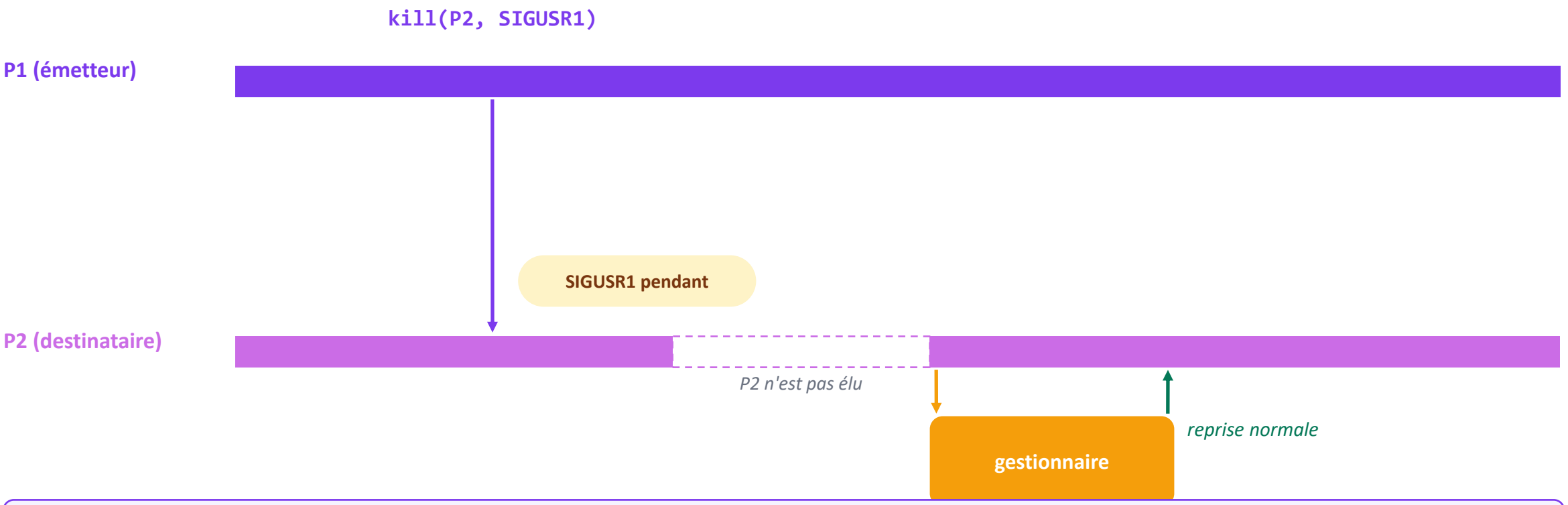
Asynchrone

Le signal n'est pas reçu au moment de l'envoi. Il est marqué « pendant » dans le processus destinataire, et délivré plus tard.

Pas de file d'attente

Si le même signal arrive deux fois avant d'être délivré, il n'est compté qu'une fois : les signaux classiques peuvent se perdre.

La délivrance d'un signal, pas à pas



Les signaux sont traités quand le destinataire repasse du mode noyau au mode utilisateur (fin d'appel système, d'interruption ou d'élection). D'où le délai entre l'envoi et la réception.

Choisir la réaction : signal

```c
#include <signal.h>

typedef void (*sighandler_t)(int);
sighandler_t signal(int sig, sighandler_t action);
```

```c
signal(SIGINT, SIG_IGN);     // Ctrl+C est ignoré
signal(SIGINT, SIG_DFL);     // retour à l'action par défaut
signal(SIGINT, quefaire);    // la fonction quefaire(int) est appelée
```

- action : SIG_IGN, SIG_DFL, ou l'adresse d'une fonction void f(int)
- Le gestionnaire reçoit le numéro du signal en paramètre
- signal renvoie l'ancienne action, ou SIG_ERR en cas d'erreur

Prototype corrigé
signal renvoie un pointeur de fonction, ce n'est pas une fonction void : c'est pour cela qu'on passe par le typedef sighandler_t.

Préférez sigaction
Le comportement de signal varie entre systèmes (System V ou BSD : le gestionnaire est-il réinstallé ? les appels système redémarrés ?). sigaction est normalisé par POSIX.

Exemple : intercepter Ctrl+C

quefaire.c

```c
#include <signal.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>

volatile sig_atomic_t recu = 0;

void quefaire(int sig)
{
    recu = sig;                 // le strict minimum
}

int main(void)
{
    signal(SIGINT, quefaire);   // Ctrl+C
    signal(SIGQUIT, SIG_IGN);   // Ctrl+\ ignoré
    printf("PID %d\n", getpid());
    for (;;) {
        pause();                // attend un signal
        printf("reçu : %s\n", strsignal(recu));
    }
}
```

```
$ ./quefaire
PID 4321
^Creçu : Interrupt
^\^Creçu : Interrupt
$ kill 4321     # depuis un autre terminal
Terminated
```

Ancienne version du cours : printf et exit dans le gestionnaire. C'est risqué : si le signal interrompt un printf du programme principal, le tampon de stdio peut être corrompu. On se contente de positionner un indicateur.

Attendre : pause et alarm

```c
#include <unistd.h>

int pause(void);                    // dort jusqu'à un signal
unsigned int alarm(unsigned int s);  // SIGALRM dans s secondes
```

decollage.c

```c
void sonnerie(int sig) { (void) sig; }  // réveille pause

int main(void)
{
    signal(SIGALRM, sonnerie);
    for (int i = 5; i > 0; i--) {
        printf("%d...\n", i);
        alarm(1);
        pause();
    }
    printf("Décollage !\n");
    return 0;
}
```

```
$ ./decollage
5...
4...
3...
2...
1...
Décollage !
```

- alarm(0) annule l'alarme en cours
- alarm renvoie le nombre de secondes qui restaient
- Si le gestionnaire est SIG_DFL, SIGALRM termine le processus

Course possible : si SIGALRM arrive avant pause, on dort pour toujours. Solution : sigsuspend (partie 03).

02

Intercepter avec sigaction

L'interface POSIX, fiable et portable

L'appel système sigaction

```
int sigaction(int sig, const struct sigaction *nouvelle,
              struct sigaction *ancienne);

struct sigaction {
    void     (*sa_handler)(int);  // SIG_DFL, SIG_IGN, f
    void     (*sa_sigaction)(int, siginfo_t *, void *);
    sigset_t   sa_mask;    // signaux bloqués pendant le gestionnaire
    int        sa_flags;   // options (voir ci-contre)
};
```

sa_flags	Effet
SA_RESTART	relance les appels système interrompus (sinon -1, errno = EINTR)
SA_SIGINFO	utilise sa_sigaction : on reçoit l'émetteur (si_pid)…
SA_RESETHAND	revient à SIG_DFL après un usage (ex SA_ONESHOT)
SA_NODEFER	ne bloque pas le signal pendant son gestionnaire (ex SA_NOMASK)

- ancienne (si non NULL) reçoit l'action précédente, pour pouvoir la restaurer
- Pendant le gestionnaire, le signal lui-même est bloqué, ainsi que ceux de sa_mask
- Le gestionnaire reste installé après usage (pas de réinstallation à faire)

Exemple : un arrêt propre

propre.c

```c
#include <signal.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>

volatile sig_atomic_t fin = 0;
void arreter(int sig) { (void) sig; fin = 1; }

int main(void)
{
    struct sigaction sa;
    memset(&sa, 0, sizeof sa);
    sa.sa_handler = arreter;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = SA_RESTART;
    sigaction(SIGINT, &sa, NULL);
    sigaction(SIGTERM, &sa, NULL);

    while (!fin) { printf("travail...\n"); sleep(1); }
    printf("arrêt propre : fichiers fermés\n");
    return 0;
}
```

```
$ ./propre
travail...
travail...
^Carrêt propre : fichiers fermés
$
```

Le schéma des serveurs : systemd, Docker ou kill envoient d'abord SIGTERM pour demander un arrêt propre (vider les tampons, supprimer les fichiers temporaires, fermer les connexions), puis SIGKILL si le programme ne s'arrête pas.

Les règles d'un bon gestionnaire

01

Court

Positionner un indicateur et revenir. Le vrai travail se fait dans le programme principal.

02

volatile sig_atomic_t

Le type des variables partagées avec le gestionnaire : lues et écrites d'un seul coup, jamais mises en cache.

03

Fonctions sûres uniquement

Seulement les fonctions « async-signal-safe » : write, _exit, kill… Pas printf, malloc, free ni exit.

04

Préserver errno

Le gestionnaire peut modifier errno au milieu du code interrompu : sauvegardez-le et restaurez-le.

03

Bloquer des signaux

Protéger une portion de code sans perdre de signal

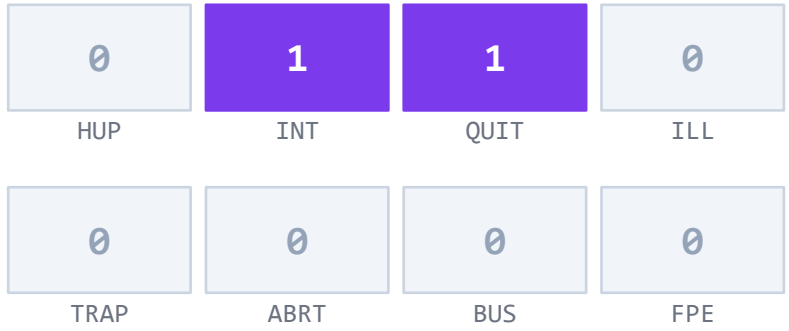
Les ensembles de signaux : sigset_t

Fonction	Effet
`sigemptyset(&e)`	vide l'ensemble
`sigfillset(&e)`	met tous les signaux dans l'ensemble
`sigaddset(&e, sig)`	ajoute un signal
`sigdelset(&e, sig)`	retire un signal
`sigismember(&e, sig)`	1 si sig appartient à l'ensemble, 0 sinon

Toutes renvoient -1 en cas d'erreur, 0 en cas de succès (sauf sigismember).

```
sigset_t e;
sigemptyset(&e);
sigaddset(&e, SIGINT);
sigaddset(&e, SIGQUIT);
```

e, vu comme un ensemble de bits

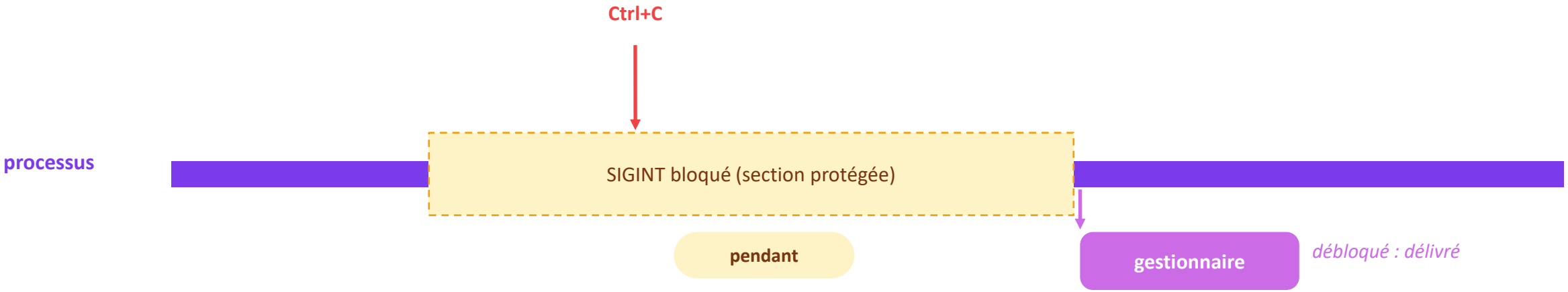


Bloquer et débloquer : sigprocmask

```
int sigprocmask(int op, const sigset_t *e, sigset_t *ancien);
```

op	Nouveau masque
SIG_BLOCK	ancien ∪ e (ajoute les signaux de e)
SIG_UNBLOCK	ancien − e (retire les signaux de e)
SIG_SETMASK	e (remplace le masque)

Bloqué n'est pas ignoré !
Un signal ignoré est perdu. Un signal bloqué reste pendant, et il est délivré dès qu'on le débloque.



Signaux pendants et sigsuspend

```
int sigpending(sigset_t *pendants);      // quels signaux attendent ?
int sigsuspend(const sigset_t *masque);  // change le masque ET attend
```

section.c

```
sigset_t bloque, ancien, pendants;
sigemptyset(&bloque);
sigaddset(&bloque, SIGINT);
sigprocmask(SIG_BLOCK, &bloque, &ancien);   // début de section

/* ... code qui ne doit pas être interrompu ... */

sigpending(&pendants);
if (sigismember(&pendants, SIGINT))
    printf("SIGINT reçu pendant la section\n");
sigprocmask(SIG_SETMASK, &ancien, NULL);    // délivré ici
```

sigpending

Donne l'ensemble des signaux bloqués qui ont été envoyés et attendent d'être délivrés.

sigsuspend

Remplace le masque et endort le processus en une seule opération atomique : plus de signal perdu entre le déblocage et l'attente (le problème de alarm + pause).

04

Les IPC

Files de messages, sémaphores, mémoire partagée

Les IPC System V

Files de messages

msgget, msgsnd, msgrcv

Sémaphores

semget, semop, semctl

Mémoire partagée

shmget, shmat, shmdt

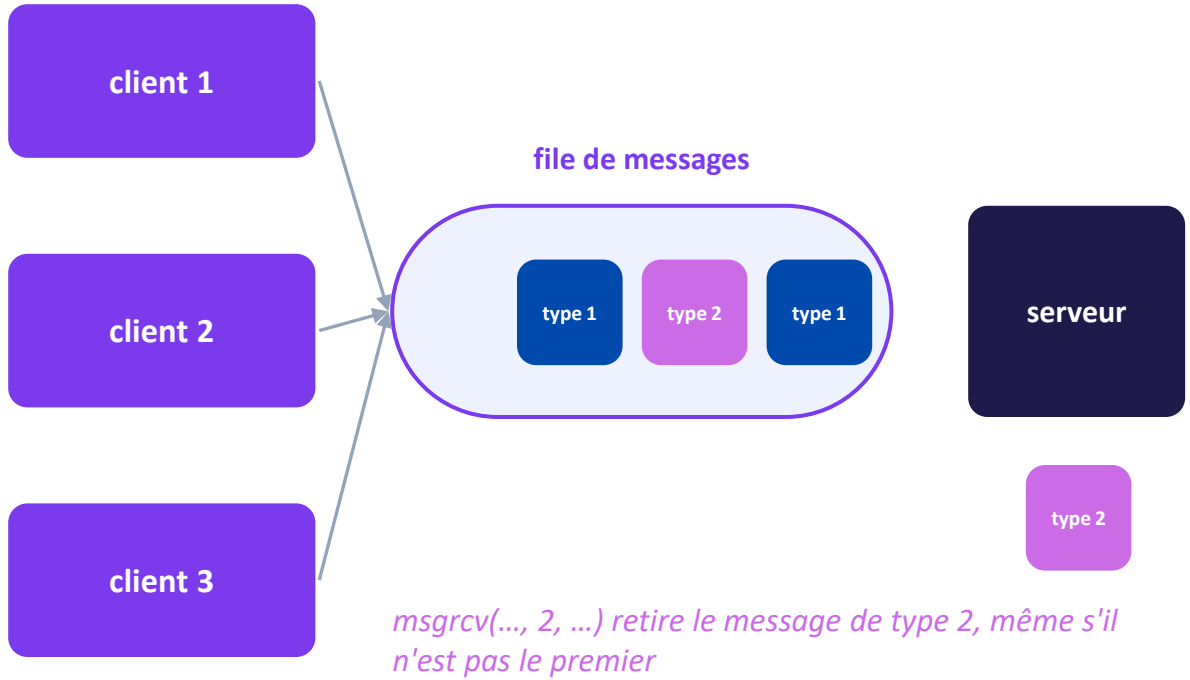


Option de création	Effet
`IPC_PRIVATE`	objet privé, réservé aux descendants
`IPC_CREAT`	crée l'objet s'il n'existe pas
`IPC_CREAT | IPC_EXCL`	erreur si l'objet existe déjà

Attention : un objet IPC survit à la fin des processus. Il faut le supprimer (…ctl avec IPC_RMID, ou la commande ipcrm). La commande ipcs les liste.

Les files de messages

- Une file FIFO de messages, gérée par le noyau
- Chaque message a un type (long > 0) : on peut retirer en priorité les messages d'un type donné
- Contrairement au tube, les frontières des messages sont conservées



msgrcv(..., 2, ...) retire le message de type 2, même s'il n'est pas le premier

file.c

```c
struct message { long type; char texte[64]; };

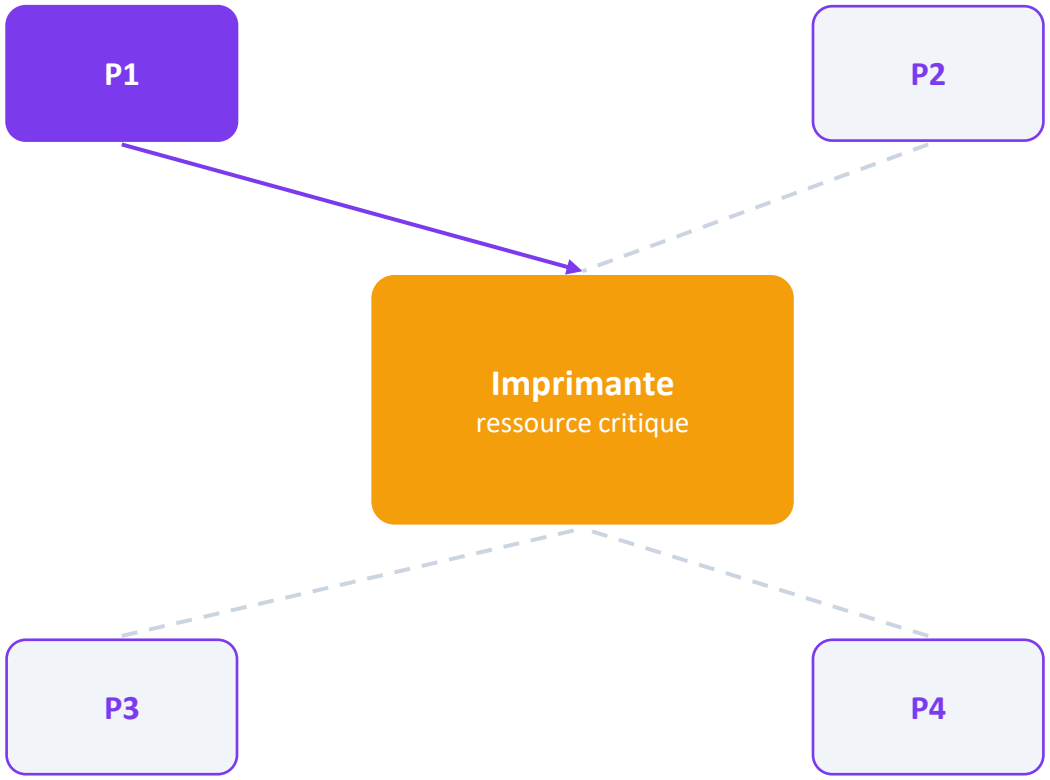
int id = msgget(cle, IPC_CREAT | 0600);
struct message m = { 1, "bonjour" };
msgsnd(id, &m, sizeof m.texte, 0);
msgrcv(id, &m, sizeof m.texte, 2, 0);  // type 2
msgctl(id, IPC_RMID, NULL);            // supprimer
```

Les sémaphores System V

- Un sémaphore protège une ressource critique partagée : une imprimante, une zone mémoire…
- semget crée un ensemble de sémaphores, semop les modifie (atomiquement), semctl les initialise ou les supprime
- Quand plusieurs processus attendent, l'ordre de réveil n'est pas garanti (pas de FIFO)

```c
int id = semget(cle, 1, IPC_CREAT | 0600);
semctl(id, 0, SETVAL, 1);            // libre

struct sembuf P = {0, -1, 0}, V = {0, +1, 0};
semop(id, &P, 1);                    // prendre
/* section critique : imprimer */
semop(id, &V, 1);                    // rendre
```

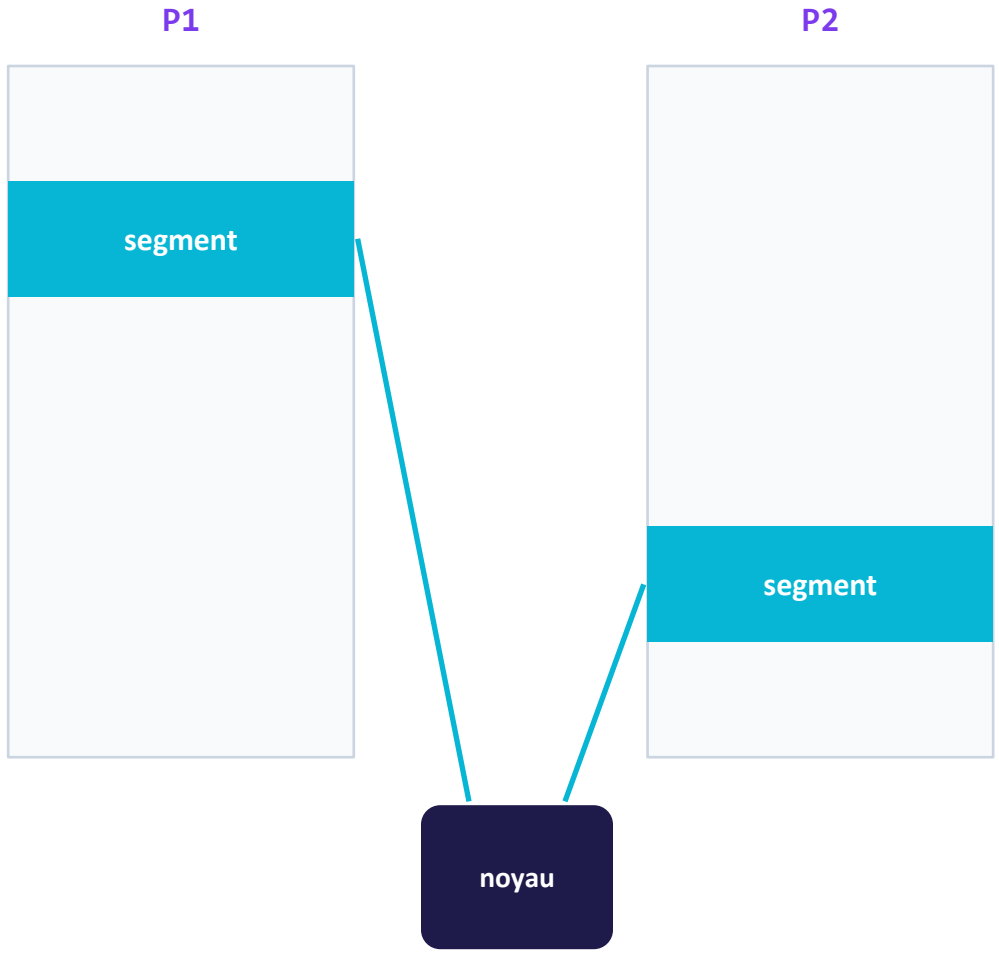


Un seul processus à la fois : les autres attendent dans semop.

La mémoire partagée

```c
int id = shmget(cle, 4096, IPC_CREAT | 0600);
int *compteur = shmat(id, NULL, 0);   // attacher
*compteur = *compteur + 1;             // accès direct !
shmdt(compteur);                       // détacher
shmctl(id, IPC_RMID, NULL);            // supprimer
```

- Le moyen de communication le plus rapide : aucune recopie, on lit et écrit directement la mémoire
- Mais aucune synchronisation : il faut un sémaphore pour éviter les conditions de concurrence (CM 03b, CM 05)
- Le segment peut être attaché à des adresses virtuelles différentes dans chaque processus : ne pas y stocker de pointeurs



System V ou POSIX ?

Mécanisme	System V	POSIX
Files de messages	msgget, msgsnd, msgrcv	mq_open, mq_send, mq_receive
Sémaphores	semget, semop, semctl	sem_open, sem_wait, sem_post
Mémoire partagée	shmget, shmat, shmdt	shm_open, ftruncate, mmap
Nommage	clé numérique (ftok)	nom « /quelque_chose »
Observer	ipcs, ipcrm	ls /dev/shm, /dev/mqueue

System V

L'interface historique (années 1980), encore très répandue. Plus lourde : clés, ensembles de sémaphores, nettoyage manuel.

POSIX

Plus simple et plus cohérente avec les fichiers (noms, descripteurs). À privilégier dans du code neuf ; compiler avec -lrt -pthread si besoin.

SYNTHÈSE

À retenir

01

Signal

Une notification asynchrone sans données. Action par défaut, ignorer ou intercepter.

02

kill

kill(pid, sig) envoie, la délivrance est différée. Les signaux classiques ne s'empilent pas.

03

sigaction

À préférer à signal. Gestionnaire court : volatile sig_atomic_t, fonctions sûres seulement.

04

Masque

Bloquer n'est pas ignorer : le signal reste pendant. sigsuspend attend sans course.

05

IPC

Files de messages, sémaphores, mémoire partagée : System V (clés) ou POSIX (noms).

06

Nettoyer

Les objets IPC survivent aux processus : IPC_RMID, ipcrm, shm_unlink.

Ressources pédagogiques

Le manuel

```
$ man 7 signal          # vue d'ensemble
$ man 7 signal-safety   # fonctions sûres
$ man 2 sigaction
$ man 2 sigprocmask
$ man 7 sysvipc
$ man 7 shm_overview
```

man 7 signal-safety donne la liste exacte des fonctions utilisables dans un gestionnaire.

Lectures

- M. Kerrisk, The Linux Programming Interface : chapitres 20 à 22 (signaux), 45 à 48 (IPC System V), 51 à 54 (IPC POSIX)
- J.-M. Rifflet, La programmation sous UNIX, Ediscience
- Guide ANSSI de programmation sécurisée en C : règles sur les gestionnaires de signaux

lamae

Questions ?

PROCHAINE SÉANCE

CM 05 — Sémaphores, moniteurs et philosophes

sleep / wakeup · producteur-consommateur · sémaphores · moniteurs · dîner des philosophes

Christophe Cruz · christophe.cruz@ube.fr